

1. Introduction

Le projet consiste à réaliser un morphing entre deux images. Le morphing est une technique qui permet de transformer progressivement une image de départ en une image d'arrivée en utilisant la triangulation.

L'objectif est de générer un ensemble d'images intermédiaires puis de créer une vidéo fluide montrant la transformation.

morphing Image_Depart Image_Arrivee N

- Image_Depart : l'image initiale (format PPM)
- Image_Arrivee : l'image finale (format PPM)
- N : le nombre d'images intermédiaires à générer

Le programme produit donc N+1 images intermédiaires et un film d'environ 10 secondes.

Il suffit d'écrire : **make** dans le terminal pour lancer le programme.

2. Outils et bibliothèques utilisés

Pour ce projet, plusieurs outils et bibliothèques ont été utilisés :

1. Langage C : pour la lecture des images, le calcul des images intermédiaires et la gestion des structures de données.
2. Bibliothèque `uvsqgraphics_2` : pour l'affichage graphique des images et des points de correspondance.
3. `FFmpeg` : pour assembler les images intermédiaires et générer une vidéo finale.
4. Format PPM (P3) : format simple d'images couleur utilisé pour sa facilité de lecture et de manipulation en C.

3. Lecture et affichage des images

3.1 Format PPM

Le format PPM stocke les images en trois composantes de couleur (R, G, B) pour chaque pixel.

- P3 : ASCII, lisible mais volumineux → utilisé dans le projet
- P6 : binaire, plus rapide et compact

3.2 Structure de données

Chaque image est stockée dans une structure contenant :

- largeur et hauteur
- la valeur maximal pour les composantes(rouge,vert,bleu)
- le nombre de pixel de decalage entre les deux images
- tableau des pixels RGB

Chaque points a sa propre strucutre selon sa provenance :

- liste des couples de points pour les images de départ et d'arriver(LISTE_POINTS)
- liste des points intermédiaires(POINTS_INT)

Et tout les triangles sont stocker dans la structure TRIANGLE_HEAD :

- les triangles de départ sont dans TH_D
- les triangles d'arriver sont dans TH_A
- les triangles intermédiaires sont dans TH.

3.3 Affichage

Les images de départ et d'arrivée sont affichées avec la bibliothèque uvsqgraphics_2.
Cela permet à l'utilisateur de sélectionner les points de base en visualisant les deux images.

4. Sélection des points de base

4.1 Points de correspondance

Pour que le morphing soit cohérent visuellement, il faut définir des points de base correspondants sur chaque image.

Chaque point de l'image de départ doit avoir un point correspondant sur l'image d'arrivée.

L'utilisateur doit d'abord cliquer sur une image puis l'autre, si il clique de fois de suite sur une même image le programme s'arrête.

4.2 Interface graphique

L'interface permet :

1. de cliquer sur un point dans l'image de départ
2. de cliquer sur le point correspondant dans l'image d'arrivée
3. d'afficher le couple de points au centre, numéroté, entouré d'un cercle

4.3 Sauvegarde

Un bouton **Sauver et trianguler** permet d'enregistrer les couples de points dans un fichier, et lancer la triangulation.

Les coins de l'image sont ajoutés automatiquement pour assurer une triangulation correcte.

Un bouton Quitter permet de fermer l'application.

5. Calcul des images intermédiaires

5.1 Interpolation des points de base

Chaque point intermédiaire est calculé par interpolation linéaire :

$$[ix_n = (1-\alpha) * dx_n + \alpha * ax_n]$$

$$[iy_n = (1-\alpha) * dy_n + \alpha * ay_n]$$

où (dx_n, dy_n) est le point de départ et (ax_n, ay_n) le point d'arrivée.

5.2 Triangulation

Les points de base sont triangulés pour savoir comment déformer chaque zone de l'image:

1. les 4 premiers points forment le rectangle englobant
2. chaque point supplémentaire subdivise un triangle existant
3. on obtient au final $2N - 6$ triangles si $N \geq 5$

5.3 Interpolation des pixels

Pour chaque pixel $P(x,y)$:

1. déterminer le triangle auquel il appartient
2. calculer ses coordonnées barycentriques $((\lambda, \mu))$:
 $[P = A + \lambda * AB + \mu * AC]$
3. retrouver la position correspondante dans l'image de départ et l'image d'arrivée
4. interpoler chaque composante couleur :
 $[rouge(P) = (1-\alpha) * rouge(P_D) + \alpha * rouge(P_A)]$

6. Sauvegarde des images

Chaque image intermédiaire est enregistrée au format PPM dans le dossier **FRAMES** :

frame_00.ppm, frame_01.ppm, ..., frame_N.ppm

Ces images serviront ensuite à créer la vidéo.

7. Création du film

La vidéo est générée à partir des images intermédiaires avec FFmpeg :

```
ffmpeg -framerate 10 -i frame_%02d.ppm morphing.mp4
```

- -framerate 10 : 10 images par seconde
- frame_%02d.ppm : les images intermédiaires
- morphing.mp4 : nom de la vidéo finale

Le résultat est une animation fluide montrant la transformation complète.

8. Conclusion

Ce projet a permis de mettre en pratique plusieurs notions :

- programmation en C et manipulation de structures complexes
- traitement d'images avec interpolation et triangulation
- utilisation d'outils externes : SDL2 et FFmpeg

Les principales difficultés ont été la gestion des structures de données comme les points de base ou les triangles de la triangulation et le calcul des couleurs interpolées.

Le résultat final est un programme fonctionnel qui crée un morphing visuel fluide entre deux images.